

Tecniche avanzate

Generics

Come usare tipi parametrizzati per collezioni e classi riutilizzabili.

Il problema che risolvono

Hai già visto codice come questo:

```
ArrayList nomi = new ArrayList<>();
```

La parte `String` tra parentesi angolari dice che la lista contiene stringhe.

Questa idea si chiama **generics**.

Tipi più sicuri

Senza generics, una lista potrebbe contenere di tutto: testi, numeri, oggetti diversi.

Con i generics, Java controlla il tipo.

```
ArrayList nomi = new ArrayList<>();

nomi.add("Luca");
nomi.add("Sara");
// nomi.add(10); // errore
```

`10` è un `int`, non una `String`. Java lo segnala subito.

Generics nelle collezioni

Esempi comuni:

```
ArrayList nomi = new ArrayList<>();
ArrayList numeri = new ArrayList<>();
HashMap prezzi = new HashMap<>();
HashSet parole = new HashSet<>();
```

`Integer` è la versione oggetto di `int`. Nelle collezioni Java si usano tipi oggetto, non primitivi.

Altri esempi:

- `Double` per `double`
- `Boolean` per `boolean`
- `Character` per `char`

Creare una classe generica

Puoi creare anche le tue classi generiche.

```
public class Scatola {
    private T valore;

    public Scatola(T valore) {
        this.valore = valore;
    }

    public T getValore() {
        return valore;
    }
}
```

`T` è un parametro di tipo. Significa: il tipo verrà scelto quando userai la classe.

Usare la classe generica

```
public class Esempio {  
    public static void main(String[] args) {  
        Scatola scatolaNome = new Scatola<>("Luca");  
        Scatola scatolaNumero = new Scatola<>(42);  
  
        System.out.println(scatolaNome.getValore());  
        System.out.println(scatolaNumero.getValore());  
    }  
}
```

Output:

```
Luca  
42
```

La stessa classe `Scatola` funziona con tipi diversi, ma resta controllata.

Perche non usare Object

Potresti pensare di usare `Object`, il tipo piu generale.

```
public class Scatola {  
    private Object valore;  
}
```

Il problema e che poi perdi informazioni sul tipo e devi fare cast manuali.

I generics evitano molti cast e molti errori.

Regola pratica

Usa i generics quando una classe o una collezione deve lavorare con tipi diversi, ma vuoi mantenere controlli chiari.

Nella maggior parte dei programmi iniziali li incontrerai soprattutto con collezioni come `ArrayList`, `HashMap` e `HashSet`.

Lambda

Come scrivere funzioni brevi da passare ad altri metodi.

A cosa servono le lambda

Una **lambda** è un modo breve per scrivere una piccola azione.

La usi quando un metodo ti chiede:

"Che cosa devo fare con questo valore?"

Per esempio, una lista può chiederti cosa fare con ogni nome. Tu puoi rispondere con una lambda:

```
nome -> System.out.println(nome)
```

Si legge così:

"prendi un `nome` e stampalo".

La freccia `->` separa due parti:

- a sinistra c'è il valore che arriva
- a destra c'è l'azione da eseguire

La forma più semplice

La forma più comune è questa:

```
parametro -> azione
```

Il `parametro` è il valore che Java passa alla lambda.

L' `azione` è il codice che vuoi eseguire con quel valore.

Esempio:

```
nome -> System.out.println(nome)
```

Qui `nome` non è una variabile creata prima da noi. È il nome che diamo al valore ricevuto dalla lambda.

Usare una lambda con `forEach`

Vediamo un esempio completo con una lista di nomi.

```
public class EsempioLambda {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        nomi.forEach(nome -> System.out.println(nome));  
    }  
}
```

Output:

```
Luca  
Sara  
Mina
```

`forEach` significa "per ciascun elemento".

Quindi questa riga:

```
nomi.forEach(nome -> System.out.println(nome));
```

vuol dire:

"per ogni nome nella lista `nomi` , stampa quel nome".

Java prende un elemento alla volta dalla lista. Ogni elemento viene chiamato `nome` dentro la lambda.

Confronto con il for-each

La stessa cosa si può scrivere anche con un ciclo `for-each` .

```
for (String nome : nomi) {  
    System.out.println(nome);  
}
```

Con una lambda diventa:

```
nomi.forEach(nome -> System.out.println(nome));
```

I due esempi fanno la stessa cosa.

All'inizio il ciclo `for-each` è spesso più facile da leggere. La lambda diventa utile quando usi metodi che vogliono ricevere una piccola azione, come `forEach` , `sort` , `filter` o `map` .

Lambda con più righe

Se l'azione è molto breve, puoi scriverla dopo la freccia.

Se invece servono più righe, usa le parentesi graffe.

```
nomi.forEach(nome -> {  
    String maiuscolo = nome.toUpperCase();  
    System.out.println(maiuscolo);  
});
```

Output:

```
LUCA  
SARA  
MINA
```

Qui succedono due cose per ogni nome:

1. Java crea una versione in maiuscolo.
2. Poi stampa il risultato.

Attenzione: se usi le parentesi graffe, il codice dentro la lambda deve essere scritto come un normale blocco Java.

Lambda che restituisce un valore

Una lambda può anche produrre un risultato.

Per esempio:

```
numero -> numero * 2
```

Si legge:

"prendi `numero` e restituisci `numero * 2`".

In una lambda breve non devi scrivere `return`.

```
Operazione doppio = numero -> numero * 2;
```

Questa lambda prende un numero e produce il suo doppio.

Interfacce funzionali

Una lambda non vive da sola. Java deve sapere che tipo di azione rappresenta.

Per questo una lambda si usa quando Java si aspetta un'interfaccia con **un solo metodo astratto**.

Questa interfaccia si chiama **interfaccia funzionale**.

Esempio:

```
public interface Operazione {  
    int esegui(int numero);  
}
```

Questa interfaccia dice:

"un'operazione prende un `int` e restituisce un `int`".

Ora possiamo creare un'operazione con una lambda.

```
Operazione doppio = numero -> numero * 2;  
  
System.out.println(doppio.esegui(5));
```

Output:

```
10
```

La lambda:

```
numero -> numero * 2
```

diventa il corpo del metodo `esegui`.

Quando chiami:

```
doppio.esegui(5)
```

Java mette `5` al posto di `numero` e calcola `5 * 2`.

Lambda con due parametri

Una lambda puo ricevere anche piu valori.

In quel caso metti i parametri tra parentesi.

```
(a, b) -> a.compareTo(b)
```

Qui la lambda riceve due valori: `a` e `b` .

Vediamola dentro `sort` , che ordina una lista.

```
public class OrdinaNomi {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Sara");  
        nomi.add("Luca");  
        nomi.add("Mina");  
  
        nomi.sort((a, b) -> a.compareTo(b));  
  
        System.out.println(nomi);  
    }  
}
```

Output:

```
[Luca, Mina, Sara]
```

`sort` deve sapere come confrontare due elementi.

La lambda:

```
(a, b) -> a.compareTo(b)
```

dice a Java:

"confronta `a` con `b` usando l'ordine naturale delle stringhe".

Quando usarle

Le lambda sono utili quando:

- l'azione è breve
- il codice resta facile da leggere
- stai passando un comportamento a un metodo
- usi collezioni, ordinamenti o stream

Evita lambda troppo lunghe. Se dentro la lambda ci sono molte righe, spesso è meglio creare un metodo con un nome chiaro.

Da ricordare

- Una lambda descrive una piccola azione.
 - La freccia `->` significa "usa questo valore per fare questa cosa".
 - A sinistra della freccia ci sono i parametri.
 - A destra della freccia c'è il codice da eseguire.
 - Le lambda sono utili quando un metodo chiede un comportamento da applicare.
-

Record ed enum

Come rappresentare dati compatti e insiemi di valori fissi.

Due strumenti per dati semplici

Java offre strumenti che rendono più comodo rappresentare alcuni dati.

In questa pagina vediamo:

- `enum`, per valori fissi
- `record`, per oggetti semplici e immutabili

Sono concetti più moderni rispetto alle basi, ma molto utili quando il modello dei dati è chiaro.

enum

Un `enum` rappresenta un insieme chiuso di valori possibili.

Esempio:

```
public enum Giorno {  
    LUNEDI,  
    MARTEDI,  
    MERCOLEDI,  
    GIOVEDI,  
    VENERDI,  
    SABATO,  
    DOMENICA  
}
```

Una variabile di tipo `Giorno` può contenere solo uno di questi valori.

```
Giorno oggi = Giorno.LUNEDI;
```

Perche usare enum

Senza enum potresti usare stringhe:

```
String stato = "pagato";
```

Ma potresti anche sbagliare:

```
String stato = "pagatto";
```

Con un enum, Java controlla i valori validi.

```
public enum StatoOrdine {  
    NUOVO,  
    PAGATO,  
    SPEDITO,  
    CONSEGNATO  
}
```

Uso:

```
StatoOrdine stato = StatoOrdine.PAGATO;
```

enum con switch

```
StatoOrdine stato = StatoOrdine.SPEDITO;  
  
switch (stato) {  
    case NUOVO:  
        System.out.println("Ordine creato");  
        break;  
    case PAGATO:  
        System.out.println("Pagamento ricevuto");  
        break;  
    case SPEDITO:  
        System.out.println("Ordine in viaggio");  
        break;  
    case CONSEGNATO:  
        System.out.println("Ordine consegnato");  
        break;  
}
```

Gli enum sono ottimi quando hai un elenco limitato e conosciuto di scelte.

record

Un **record** serve a rappresentare dati semplici.

```
public record Prodotto(String nome, double prezzo) {  
}
```

Questa riga crea automaticamente:

- campi privati e finali
- costruttore
- metodi per leggere i valori
- `toString`
- `equals`
- `hashCode`

Usare un record

```
public class EsempioRecord {  
    public static void main(String[] args) {  
        Prodotto prodotto = new Prodotto("Quaderno", 2.50);  
  
        System.out.println(prodotto.nome());  
        System.out.println(prodotto.prezzo());  
        System.out.println(prodotto);  
    }  
}
```

Output:

```
Quaderno  
2.5  
Prodotto[nome=Quaderno, prezzo=2.5]
```

I metodi di lettura hanno lo stesso nome dei campi: `nome()` e `prezzo()`.

Record immutabili

Un record è pensato per dati immutabili: dopo la creazione, non cambi i campi.

Non fai:

```
prodotto.prezzo = 3.00; // non valido
```

Se ti serve un prodotto con prezzo diverso, crei un nuovo record.

Quando usarli

Usa **enum** quando un valore può essere scelto da un insieme fisso:

- giorni della settimana
- stati di un ordine
- livelli di priorità

Usa **record** quando vuoi trasportare dati semplici senza scrivere una classe lunga:

- prodotto con nome e prezzo
- punto con x e y
- persona con nome ed età

Se l'oggetto deve cambiare spesso o avere molta logica interna, una classe normale può essere più adatta.

Stream

Come trasformare e filtrare collezioni con una sintassi dichiarativa.

Cosa sono gli stream

Uno stream è un modo per lavorare su una sequenza di dati dichiarando cosa vuoi ottenere.

Invece di scrivere ogni passaggio con un ciclo, componi operazioni come:

- filtra
- trasforma
- raccogli il risultato

Gli stream non sostituiscono sempre i cicli. Sono uno strumento in più, utile quando il flusso di trasformazione è chiaro.

Da lista a stream

```
public class EsempioStream {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        List risultato = nomi.stream()  
            .filter(nome -> nome.length() > 4)  
            .toList();  
  
        System.out.println(risultato);  
    }  
}
```

Output:

```
[Luca, Sara]
```

Mina ha 4 caratteri, quindi viene esclusa dalla condizione `> 4`.

Collezione e stream

Una collezione, come `ArrayList`, contiene davvero i dati.

Uno stream è un flusso di lavoro su quei dati.

```
nomi.stream()
```

crea uno stream a partire dalla lista.

Alla fine, con `toList()`, raccogli il risultato in una nuova lista.

filter

`filter` tiene solo gli elementi che rispettano una condizione.

```
List numeri = List.of(3, 8, 1, 10, 5);

List grandi = numeri.stream()
    .filter(numero -> numero >= 5)
    .toList();

System.out.println(grandi);
```

Output:

```
[8, 10, 5]
```

La lambda deve restituire `true` o `false`.

map

`map` trasforma ogni elemento.

```
List nomi = List.of("luca", "sara", "mina");

List maiuscoli = nomi.stream()
    .map(nome -> nome.toUpperCase())
    .toList();

System.out.println(maiuscoli);
```

Output:

```
[LUCA, SARA, MINA]
```


Il numero di elementi resta lo stesso, ma ogni elemento viene trasformato.

Combinare filter e map

```
List nomi = List.of("luca", "sara", "anna", "mario");

List risultato = nomi.stream()
    .filter(nome -> nome.length() == 4)
    .map(nome -> nome.toUpperCase())
    .toList();

System.out.println(risultato);
```

Output:

```
[LUCA, SARA, ANNA]
```

Prima filtri i nomi lunghi 4 caratteri. Poi li trasformiamo in maiuscolo.

Quando usare uno stream

Uno stream è adatto quando stai descrivendo una trasformazione sui dati.

Un ciclo normale può essere migliore quando:

- la logica ha molti passaggi
- devi modificare variabili esterne
- il codice con stream diventa troppo difficile da leggere

Regola pratica

Se puoi leggere la catena ad alta voce, lo stream è probabilmente chiaro:

"prendi i nomi, tieni quelli lunghi 4, trasformali in maiuscolo, raccoglili in una lista".

Se devi fermarti a decifrare ogni pezzo, usa un ciclo più esplicito.